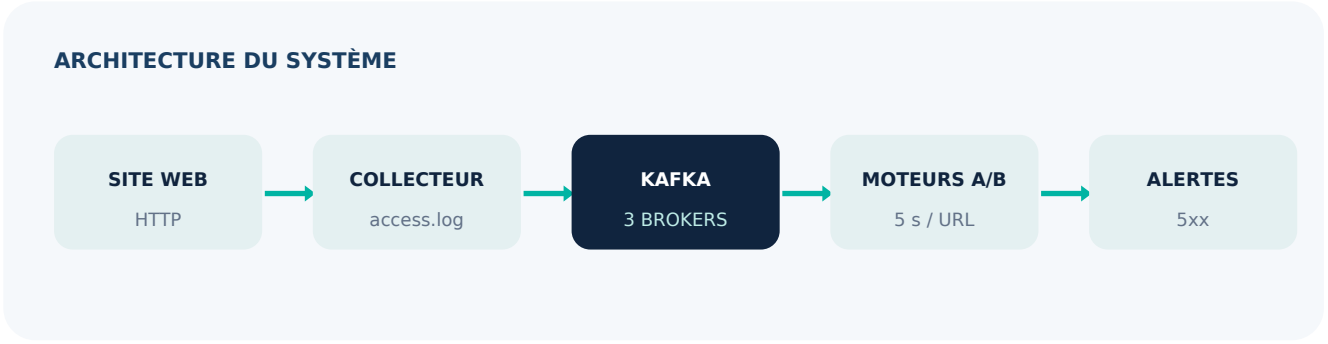
SOUKLINE

PLATEFORME DE SUPERVISION DISTRIBUÉE

RAPPORT ACADÉMIQUE / GROUPE 01

Traitement parallèle en streaming de logs Web

Architecture Kafka · Agrégation temps réel · Détection des erreurs HTTP



ÉQUIPE PROJET

Mohamed Azil

Ali Nouar

Bilal Jellaoui

Ayoub Echihami

Youssef Benayad

Souhail El Bettachi

ENCADRANT

Pr. Ayoub Allali

Université Ibn Tofaïl

Master Big Data, IA & Applications Avancées

Calcul parallèle et systèmes distribués

OCTOBRE 2026

Rapport consolidé des contributions des six membres et des résultats expérimentaux documentés.

01 · VUE D’ENSEMBLE

Résumé exécutif

SoukLine est une plateforme expérimentale de surveillance des accès web. Un site e-commerce Spring Boot produit de véritables lignes de journal HTTP. Un collecteur Java les publie dans un cluster Kafka de trois brokers. Deux moteurs indépendants — Kafka Streams et concurrence Java manuelle — calculent, par URL et par fenêtre de cinq secondes, le volume de requêtes et les erreurs HTTP 5xx. Un tableau de bord console compare les résultats et déclenche une alerte en cas de dégradation.

Les chiffres clés du projet

3	8	5 s	6
Brokers Kafka	Partitions web-logs	Fenêtre d’agrégation	Contributeurs

Apports principaux

• Pipeline de bout en bout fondé sur des requêtes HTTP réelles, et non sur des lignes de logs simulées.

• Contrat commun entre les deux moteurs : engine, windowStart, url, requests, errors5xx.

• Mode panne reproductible sur /checkout, avec réponse HTTP 503.

• Séparation entre validation fonctionnelle, tests unitaires et benchmark de performance.

Résultat principal. Les scénarios de démonstration décrivent des statistiques A/B concordantes ; les tests d’intégration réalisés sur cinq requêtes réelles montrent également 5 requêtes et 0 erreur pour chacun des deux moteurs. Les mesures comparatives de débit/latence des moteurs réels ne sont pas encore documentées dans les rapports fournis.

Lecture du rapport

Après l’architecture et la répartition des responsabilités, le document présente les composants, la méthode de validation, les résultats, les limites et les perspectives. Les valeurs de test provenant de jeux de données différents ne sont pas additionnées.

02 · CONTEXTE

Problématique et objectifs

Problématique

Dans une application web, les logs d’accès permettent de repérer les URL les plus sollicitées et les erreurs serveur. Leur traitement en flux impose toutefois une collecte continue, une messagerie résiliente, une agrégation concurrente exacte et une restitution compréhensible. La difficulté centrale est d’obtenir les mêmes statistiques avec deux approches de calcul différentes, sans confondre l’heure du log et l’heure de publication dans Kafka.

Objectifs fonctionnels

• Observer en continu les requêtes reçues par un vrai site e-commerce.

• Compter les requêtes et les erreurs 5xx par URL sur des fenêtres de cinq secondes.

• Comparer les résultats de deux moteurs et signaler un taux d’erreurs anormal.

• Reproduire des phases de trafic normal, de pic de charge et de panne avec JMeter.

Objectifs techniques

• Déployer un cluster Kafka KRaft à trois brokers avec réplication.

• Utiliser un parseur de logs partagé et une normalisation commune des URL.

• Confronter Kafka Streams à ExecutorService / ConcurrentHashMap / AtomicLong / CyclicBarrier.

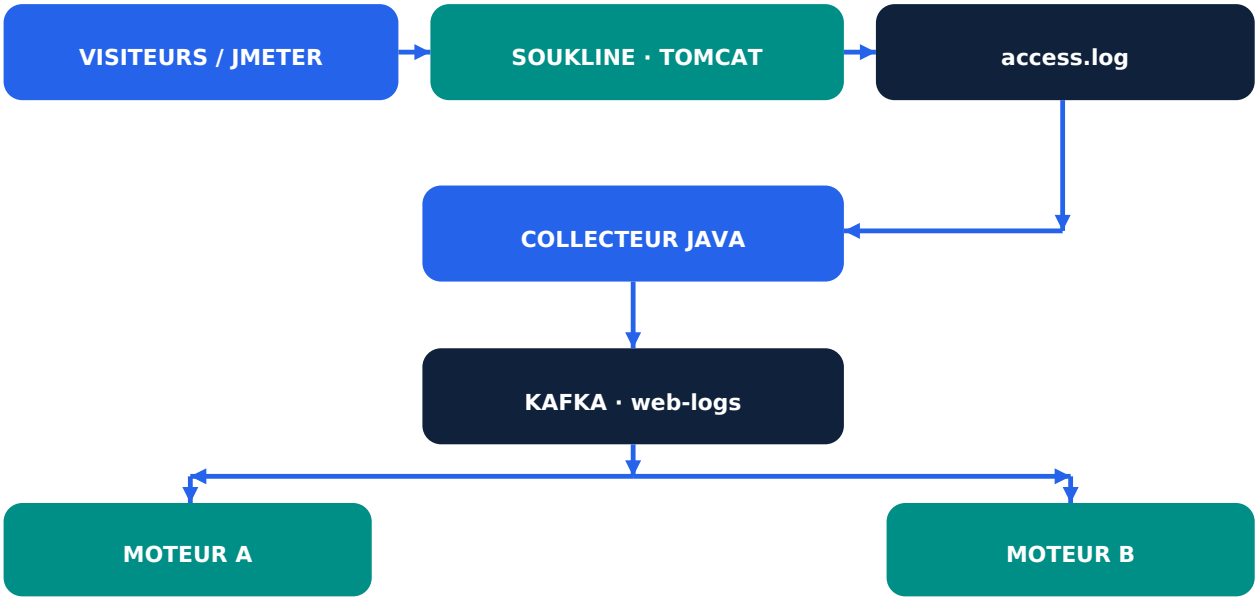
• Tester la justesse des résultats avant d’interpréter des performances.

Périmètre expérimental

Les démonstrations et les tests d’intégration portent sur des scénarios contrôlés en environnement local. Ils établissent la faisabilité fonctionnelle ; ils ne constituent pas une preuve de montée en charge en production.

03 · FLUX DE DONNÉES

Architecture globale du système



Les deux moteurs consomment les mêmes événements. Les résultats sont publiés dans stats-a et stats-b, puis comparés et affichés avec un système d'alerte.

Contrats de données

Interface	Format / propriété	Rôle
Site → collecteur	site/logs/access.log · Tomcat Common	Source HTTP réelle
Collecteur → Kafka	web-logs · 8 partitions	Transport des logs bruts
Moteurs → Kafka	stats-a et stats-b · JSON	Agrégats par fenêtre et URL
Kafka → affichage	Dernière valeur par (moteur, fenêtre, URL)	Comparaison et alertes

Sémantique des résultats

Les messages de sortie sont des totaux actualisés pour une fenêtre et une URL. Lorsque plusieurs messages concernent la même case, seul le plus récent doit être conservé ; additionner les mises à jour provoquerait un double comptage.

04 · COLLABORATION

Organisation du travail

Membre	Responsabilité	Livrable principal
1 · Mohamed Azil	Site SoukLine, logs, mode panne	site/
2 · Ali Nouar	Kafka, JMeter, scripts, intégration	infra/
3 · Bilal Jellaoui	Moteur A, parseur partagé	engine-a/ · common/
4 · Ayoub Echihami	Moteur B, concurrence manuelle	engine-b/
5 · Youssef Benayad	Collecteur, dashboard, alertes	collector-display/
6 · Souhail El Bettachi	Vérification, benchmark, rapport	verification-benchmark/

Méthode de coordination

L’équipe s’appuie sur un dépôt Maven multi-modules, des contrats d’interface communs et des branches Git par fonctionnalité. Les contributions sont proposées via Pull Request, avec revue par un binôme et tests Maven automatisés dans GitHub Actions, selon la documentation générale du dépôt.

Dépendances entre membres

• Le site fixe le format du journal et les routes utilisées par JMeter.

• Le parseur commun garantit une interprétation cohérente des logs pour A et B.

• L’infrastructure Kafka relie les producteurs, les moteurs et l’affichage.

• Le vérificateur confronte les résultats des moteurs à une référence séquentielle.

Point de vigilance documentaire : le document de répartition décrit les missions prévues ; les rapports individuels décrivent les réalisations et les essais effectivement déclarés.

05 · MEMBRE 1 — MOHAMED AZIL

Source HTTP : le site SoukLine

Le site est une application e-commerce d’artisanat marocain développée avec Spring Boot 3.3.4 et Tomcat embarqué. Il comprend un catalogue de dix produits, la recherche, le panier, le paiement simulé et une interface d’administration. Le rendu serveur Thymeleaf permet de produire de véritables accès HTTP dans le journal Tomcat.

Instrumentation et simulation de panne

• AccessLogValve écrit les requêtes dans access.log au format Common Log Format.

• La désactivation du buffering permet une lecture rapide des nouvelles lignes par le collecteur.

• L’endpoint /admin/panne?etat=on|off active ou désactive la panne du paiement.

• Lorsque la panne est active, /checkout répond HTTP 503 ; les ressources inexistantes doivent renvoyer 404, et non 500.

Résultats déclarés par le membre

Vérification	Résultat rapporté
Tests du module site	9 / 9 réussis
URL inexistante	404 NOT FOUND
Paiement en mode panne	503 SERVICE UNAVAILABLE
Navigation / panier	302 puis 200 selon le parcours

La correction du traitement des 404 empêche des alertes 5xx artificielles ; elle améliore directement la qualité des statistiques produites en aval.

06 · MEMBRE 2 — ALI NOUAR

Infrastructure et trafic de charge

L’infrastructure repose sur Apache Kafka 3.7.0, déployé par Docker Compose avec trois brokers en mode KRaft. Les ports externes utilisés sont 9098, 9094 et 9096. Le topic web-logs comporte huit partitions ; les topics stats-a et stats-b reçoivent les agrégats des deux moteurs. Des scripts PowerShell facilitent la création des topics et le contrôle de l’état du cluster.

Stratégie de génération de trafic

Phase	Scénario	Ce qui est observé
Normal	Navigation / consultation du catalogue	Requêtes 2xx, débit nominal
Pic	Charge accrue sur /panier et /checkout	Évolution du volume de logs
Panne	Activation de /admin/panne et appels /checkout	Réponses 503 et alertes 5xx

Résultats documentés

• Le README général décrit une campagne JMeter en trois phases et signale 5 000 réponses HTTP 503 durant la phase de panne.

• La démonstration intégrée décrit le passage d’un état normal à une alerte, puis le retour à la normale.

• Le test de bout en bout sur /produits a produit cinq requêtes réelles, reçues par les deux moteurs avec zéro erreur 5xx.

Interprétation : les 5 000 réponses 503 proviennent de la campagne JMeter rapportée dans la documentation du projet. Les cinq requêtes /produits constituent un test d’intégration distinct et ne doivent pas être confondues avec la charge JMeter.

07 · MEMBRES 3 ET 4

Deux moteurs, un même contrat

Moteur A — Kafka Streams (Bilal Jellaoui)

Le moteur A lit web-logs, parse les lignes avec le module common, normalise les URL, groupe les événements par clé et agrège les compteurs dans des fenêtres de cinq secondes. Le correctif LogTimeExtractor fait correspondre l'horodatage de Kafka Streams à celui du log, pour éviter les décalages de fenêtre observés durant l'intégration.

Moteur B — Concurrence Java (Ayoub Echihami)

Le moteur B utilise un ensemble de workers pilotés par ExecutorService. Chaque worker possède son KafkaConsumer ; les compteurs partagés reposent sur ConcurrentHashMap et AtomicLong. Une CyclicBarrier coordonne les instants de publication. Le moteur propose également un mode mémoire, utile pour valider les calculs sans broker Kafka.

Dimension	Moteur A	Moteur B
Approche	Topologie Kafka Streams	Workers Java explicites
Parallélisme	Stream threads / partitions	ExecutorService / consumers
État	State store / agrégation	ConcurrentHashMap + AtomicLong
Synchronisation	Gestion Kafka Streams	CyclicBarrier
Sortie	stats-a	stats-b

Contrat commun de sortie

JSON : {"engine":"A|B", "windowStart":1792270115000, "url":"/produits", "requests":5, "errors5xx":0}

08 · MEMBRE 5 — YOUSSEF BENAYAD

Collecte, tableau de bord et alerte

Collecte continue

Le collecteur Java surveille access.log à la manière de tail -f et transmet les lignes brutes au topic web-logs. Le parseur reste partagé entre les deux moteurs. Le résumé du module décrit la prise en charge des lignes incomplètes, de la troncature ou rotation du fichier, de l’absence initiale du fichier, des fins de ligne Windows et des caractères multioctets.

Visualisation

DisplayApp consomme stats-a et stats-b. StatsJson décode les messages, StatsStore retient les derniers agrégats par moteur, fenêtre et URL, et Dashboard affiche les volumes, le débit, le pourcentage d’erreurs et les URL les plus sollicitées. L’actualisation est annoncée toutes les deux secondes.

Détection d’anomalie

Règle par défaut : alerte lorsque le taux d’erreurs 5xx dépasse 10 %, avec au moins 10 requêtes. Le moteur A est prioritaire et le moteur B peut servir de repli.

Validation du module

Le résumé de Youssef recense 34 tests : LogTailer (8), StatsJson (7), StatsStore (8), AlertMonitor (6) et Dashboard (5). Il précise que les valeurs d’exemple du dashboard sont illustratives et ne doivent pas être traitées comme des mesures expérimentales.

09 · MEMBRE 6 — SOUHAIL EL BETTACHI

Vérification et benchmark CPU

Le module verification-benchmark contrôle les agrégats par URL et par fenêtre et évalue les performances de différentes stratégies de comptage parallèle.

Validation automatisée

36 / 36 tests réussis

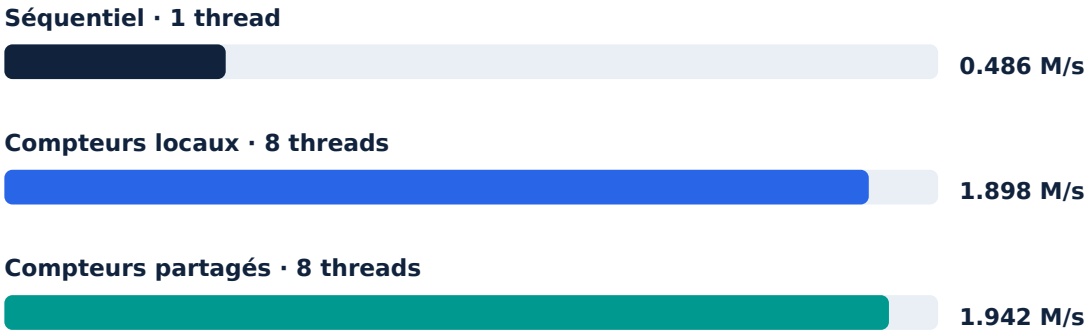
9 tests common + 27 tests verification-benchmark · BUILD SUCCESS

Protocole expérimental

62 811 lignes HTTP · 8 CPU logiques · 5 mesures · 2 échauffements

Comparaison : référence séquentielle, compteurs locaux et compteurs partagés.

Débit mesuré (millions de lignes / seconde)



Résultats comparatifs

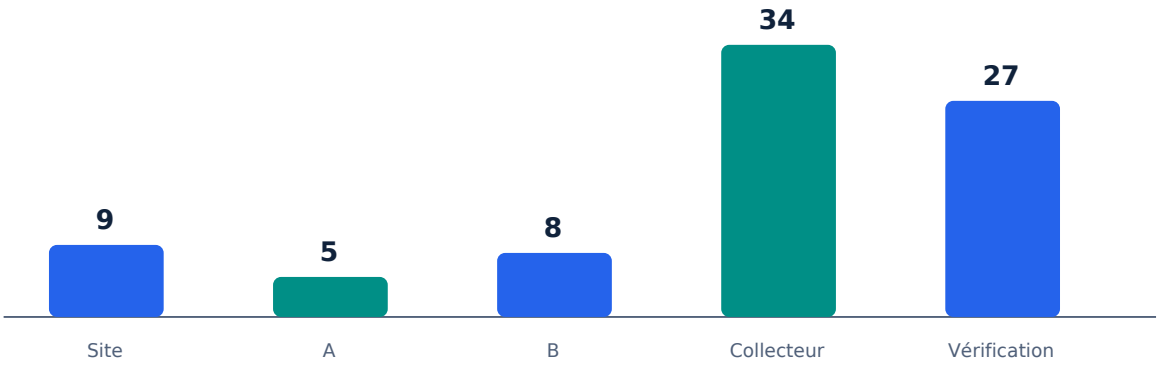
Configuration	Threads	Temps médian	Accélération
Baseline séquentielle	1	0,129 s	×1,00
Compteurs locaux	2	0,080 s	×1,62
Compteurs locaux	4	0,051 s	×2,55
Compteurs locaux	8	0,033 s	×3,91
Compteurs partagés	8	0,032 s	×4,00

Résultat : accélération ×4,00 avec 8 threads

Débit maximal : 1 942 112 lignes/s · 9 configurations correctes.

Résultats : exactitude et intégration

Tests déclarés par module



Nombre de tests déclarés dans les rapports des modules. Les 34 tests du collecteur sont décrits dans son résumé ; leur exécution n’est pas démontrée par une sortie Maven jointe.

Concordance A / B — test HTTP réel

Indicateur	Moteur A	Moteur B
URL	/produits	/produits
Fenêtre	1792270115000	1792270115000
Requêtes	5	5
Erreurs 5xx	0	0

Scénario contrôlé d’erreurs

Une injection de cinq logs sur /test-ali-integration, dont deux réponses HTTP 500, a donné 5 requêtes et 2 erreurs 5xx sur les deux moteurs pour la même fenêtre. Cette validation est distincte de la navigation réelle sur le site.

Autres essais de justesse

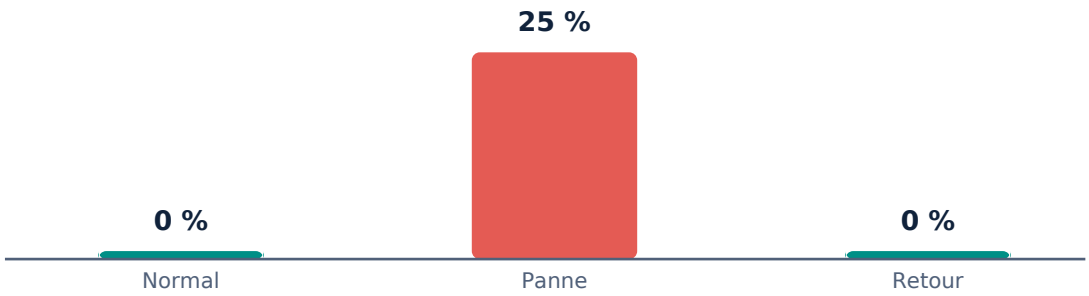
Le rapport d’Ayoub décrit un fichier de 5 000 requêtes valides avec 1 018 erreurs 5xx, correctement comptées en mode mémoire et Kafka, ainsi qu’un stress test de huit millions d’incréments sans perte. Ces résultats concernent le moteur B, et non une comparaison de débit entre A et B.

Résultats : démonstration et alertes

Démo documentée dans le README général

Situation	Moteur A	Moteur B	Lecture
Trafic normal	40 req. · 0 erreur	40 req. · 0 erreur	État OK
Panne /checkout	40 req. · 10 erreurs	40 req. · 10 erreurs	25 % · ALERTE
Retour normal	12 req. · 0 erreur	12 req. · 0 erreur	État OK

Taux d’erreurs observé pendant la panne



La démonstration indique une alerte au-dessus du seuil de 10 %, puis un retour à 0 % après désactivation du mode panne. Les publications des moteurs peuvent arriver à quelques secondes d’intervalle ; la comparaison doit retenir les valeurs finales de chaque fenêtre.

Limite : ces valeurs proviennent de la démonstration décrite dans le README général. Elles ne sont pas des statistiques globales de la campagne JMeter à 5 000 réponses 503.

Difficultés techniques et solutions

Difficulté	Solution mise en œuvre
Décalage des fenêtres A/B	TimestampExtractor fondé sur l’heure inscrite dans le log
Erreurs 500 artificielles sur routes absentes	Traitement explicite des 404 dans Spring Boot
Latence de journalisation Tomcat	Désactivation du buffering access.log
Collecte de lignes partielles / rotation	LogTailer avec gestion des cas limites
Concurrence et compteurs perdus	AtomicLong / ConcurrentHashMap et synchronisation
Risque de double comptage des résultats	Dernière valeur par fenêtre et URL
Alertes sur échantillon trop faible	Seuil minimum de 10 requêtes
Démarrage du cluster / rebalance	Attendre stabilisation KRaft et état RUNNING

Lecture critique

Les résultats de justesse sont solides sur les scénarios décrits, mais les sources n’établissent pas encore de comparaison de débit et de latence A/B en conditions identiques. Certaines contributions ont été rédigées à des moments différents du développement ; leurs observations peuvent donc refléter des versions successives du code.

13 · BILAN

Conclusion et perspectives

SoukLine réunit un site e-commerce instrumenté, un cluster Kafka à trois brokers, un collecteur continu, deux moteurs de traitement parallèle, un tableau de bord et un dispositif d'alerte. Les scénarios réalisés confirment la cohérence des agrégations et la détection des erreurs HTTP 5xx.

Réalisations majeures

- Architecture distribuée Kafka et traitement en fenêtres de cinq secondes.
- Agrégation concordante des moteurs A et B sur les scénarios d'intégration.
- Génération de trafic JMeter et simulation de pannes HTTP 503.
- 36 tests unitaires validés sur common et verification-benchmark.
- Benchmark CPU sur 62 811 logs : jusqu'à 1 942 112 lignes/s et ×4,00.

Perspectives de développement

- Approfondir les mesures de performance sous différentes charges JMeter.
- Enrichir les tableaux de bord et les indicateurs de supervision.
- Étendre les scénarios de résilience et d'automatisation des essais.

Conclusion générale

Le projet démontre une chaîne complète de supervision distribuée.
La validation fonctionnelle et le benchmark mettent en valeur
la fiabilité du traitement et les gains du calcul parallèle.

14 · RÉFÉRENTIEL DU PROJET

Dépôt GitHub officiel

Code source, modules, documentation et historique de développement.

DÉPÔT SOURCE

Bilal-Jellaoui / streaming-logs-kafka

https://github.com/Bilal-Jellaoui/streaming-logs-kafka

Contenu du dépôt

- **Application web** — Site SoukLine et génération des logs HTTP
- **Infrastructure** — Cluster Kafka, scripts et scénarios JMeter
- **Traitement** — Moteurs A et B, parseur commun
- **Supervision** — Collecteur, affichage et alertes
- **Validation** — Tests, vérification et benchmark